Studyline External Design Document

Version 1 October 30 2026

Studyline is a student assignment tracker built with MongoDB, Express, React and Node.js. It gives each registered student a private workspace for assignment details, due dates, priorities and completion status.

Team Group 2

Hamzah Al-Samadi, Kevin Mai, Rihab Chowdhury, Vitaliy Kulesh, Ebana Almahamid

Contents

1 Purpose and release scope

2 User flows and interface design

3 Data design and API architecture

4 Security and validation

5 Working interface screenshots

6 Verification and release notes

1 Purpose and release scope

The first release provides registration, login/logout, authenticated assignment CRUD, validation and ownership protection. The React interface also includes a dashboard, search, status filters, due-date and priority sorting, overdue counts, and responsive dialogs. No reminders, shared assignments, email verification or password recovery are included.

2 User flows and interface design

A visitor opens the sign-in page or creates an account. After authentication, the student sees summary counts and an assignment list. New assignment opens a focused form. Edit uses the same form with current values. Status controls support completion and reopening. Delete asks for confirmation before removing a record. Sign out clears the session.

Screen structure

Screen	Primary content	Main actions
Sign in and registration	Email, password, and name for registration	Sign in, create account, switch form
Assignment dashboard	Sidebar views, counters, search, filters, assignment cards	Create, edit, complete/reopen, delete, sign out
Assignment form	Title, course, due date, description, status, priority	Save, cancel
Delete confirmation	Assignment title and consequence	Keep assignment or confirm delete

Interface wireframes

Early layout reference for the main flows. These structural wireframes show controls and content hierarchy before visual styling.

Sign in and register

Full name (registration only)

Email address

Password

[Sign in / Create account]

Assignment dashboard

Total In progress Done Overdue

[Search] [Status] [Sort]

[+ New assignment]

[ ] Course Assignment title [Edit]

Due date Priority Status [Delete]

Create and edit

Title

Course Due date

Description

Status Priority

[Cancel] [Save]

Delete confirmation

Delete assignment?

Assignment title

This cannot be undone.

[Keep assignment] [Delete]

3 Data design and API architecture

React calls same-origin Express JSON endpoints. Routes apply authentication and validation, then delegate to controllers. Controllers call Mongoose models connected to the Atlas database student_assignment_tracker.

Collection	Fields	Constraints
users	name, email, passwordHash, createdAt, updatedAt	Unique normalized email; password hash hidden by default
assignments	owner, title, course, description, dueDate, status, priority, timestamps	Owner reference; owner/dueDate index; enum and length limits

API groups

POST /api/auth/register and /login issue an eight-hour JWT cookie. GET /api/auth/me returns public profile data. POST /api/auth/logout clears the cookie. GET/POST /api/assignments list/create records. GET/PATCH/DELETE /api/assignments/:id read/update/delete one owned record. GET /api/health reports database readiness. See API.md and the Postman collection for examples and response codes.

4 Security and validation

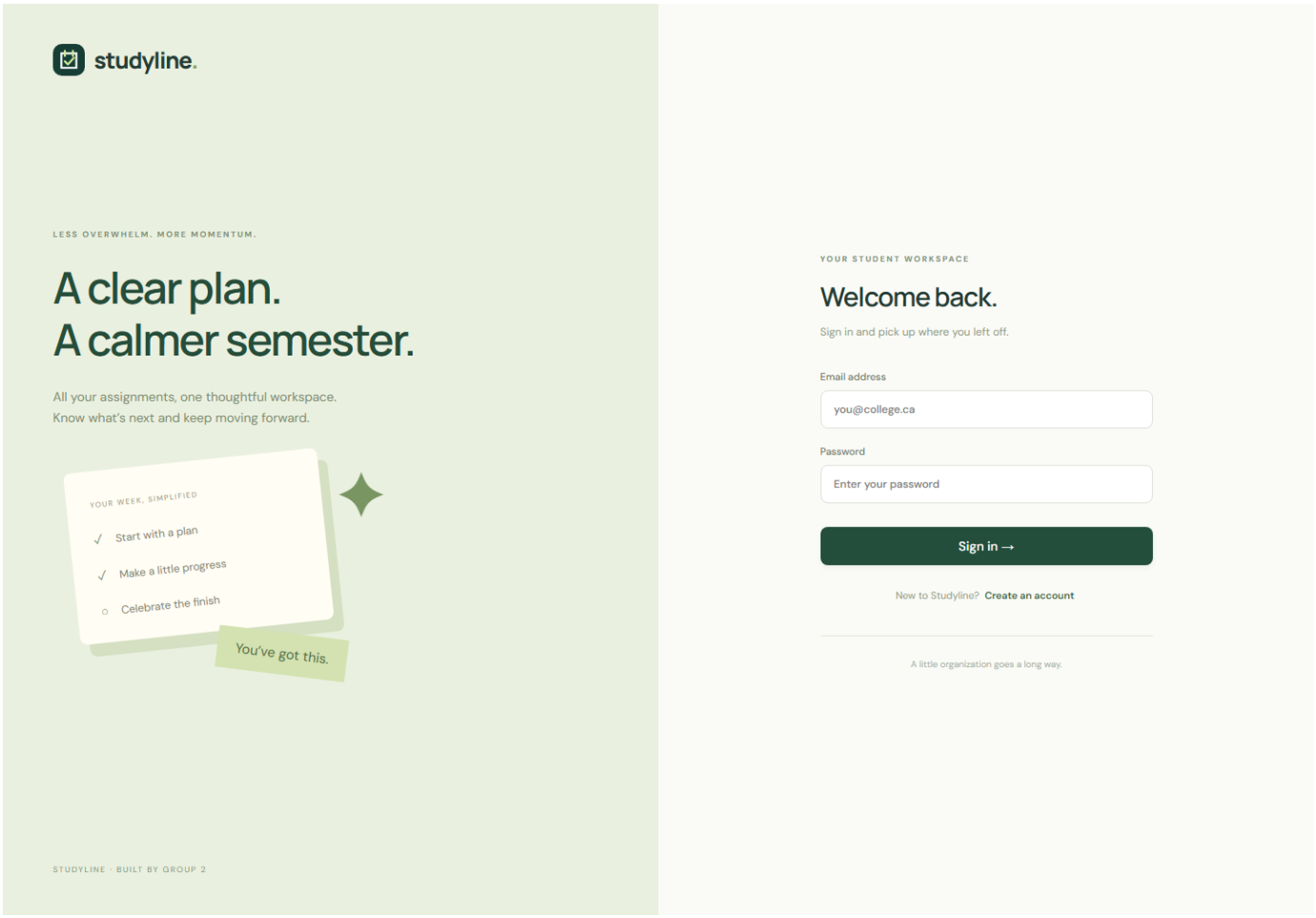
Bcrypt hashes passwords. HttpOnly and SameSite=Strict cookies limit script access and cross-site session use. Production requires HTTPS and Secure cookies. Every assignment query includes the authenticated owner ID. Unknown fields, ownership injection, invalid IDs/dates and unsupported enums are rejected. A required custom request header and origin checks protect write routes. Authentication endpoints are rate limited. Secrets are excluded from Git and archives.

Accessibility and responsive behavior

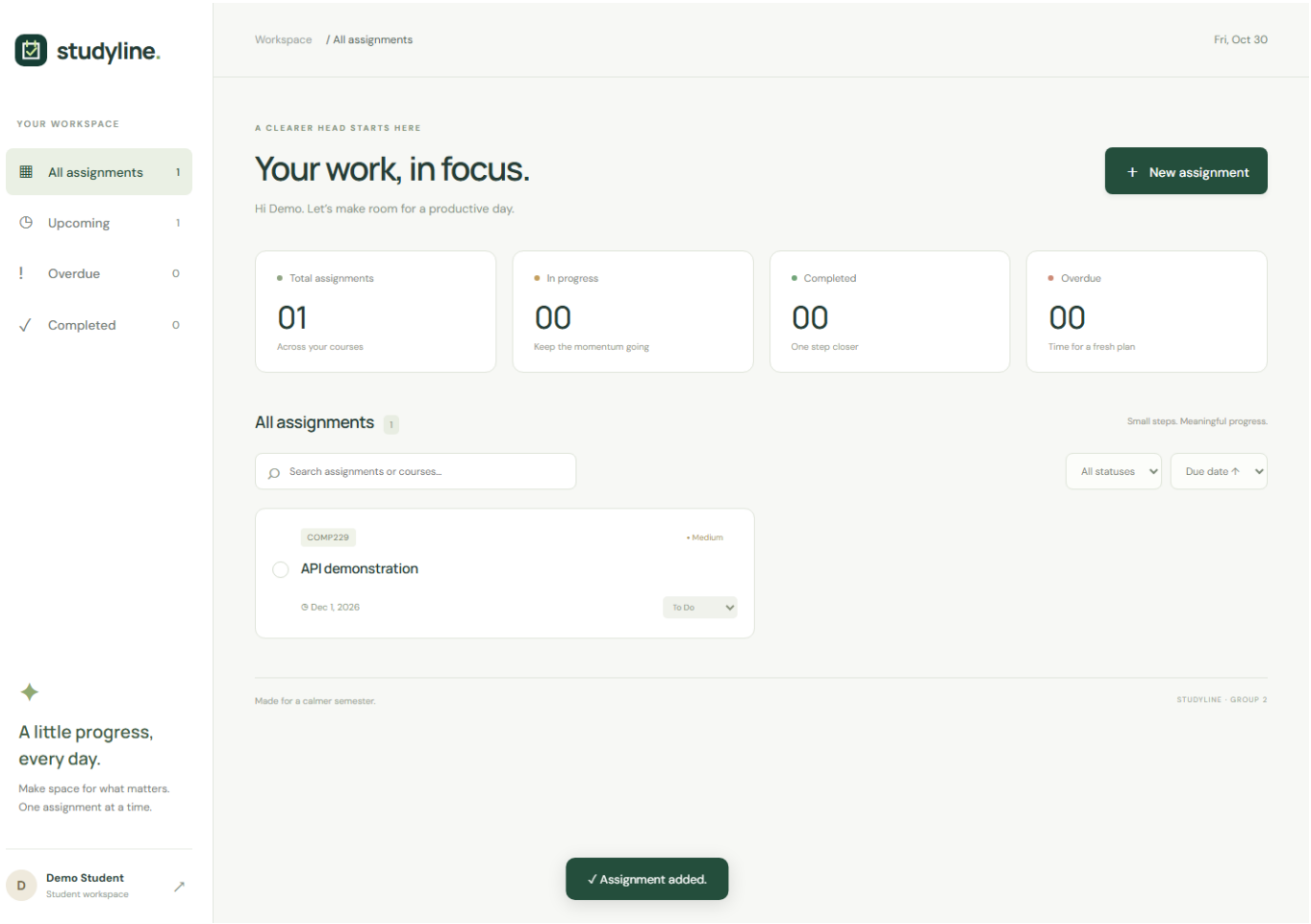
Forms have visible labels and native controls. Dialogs trap keyboard focus, restore focus when closed, and support Escape. Status and priority have text labels as well as color. Mobile layouts stack the sidebar, summary cards and assignment cards. Error and success messages use alert/status roles. Browser screenshots below document the actual implementation.

5 Working interface screenshots

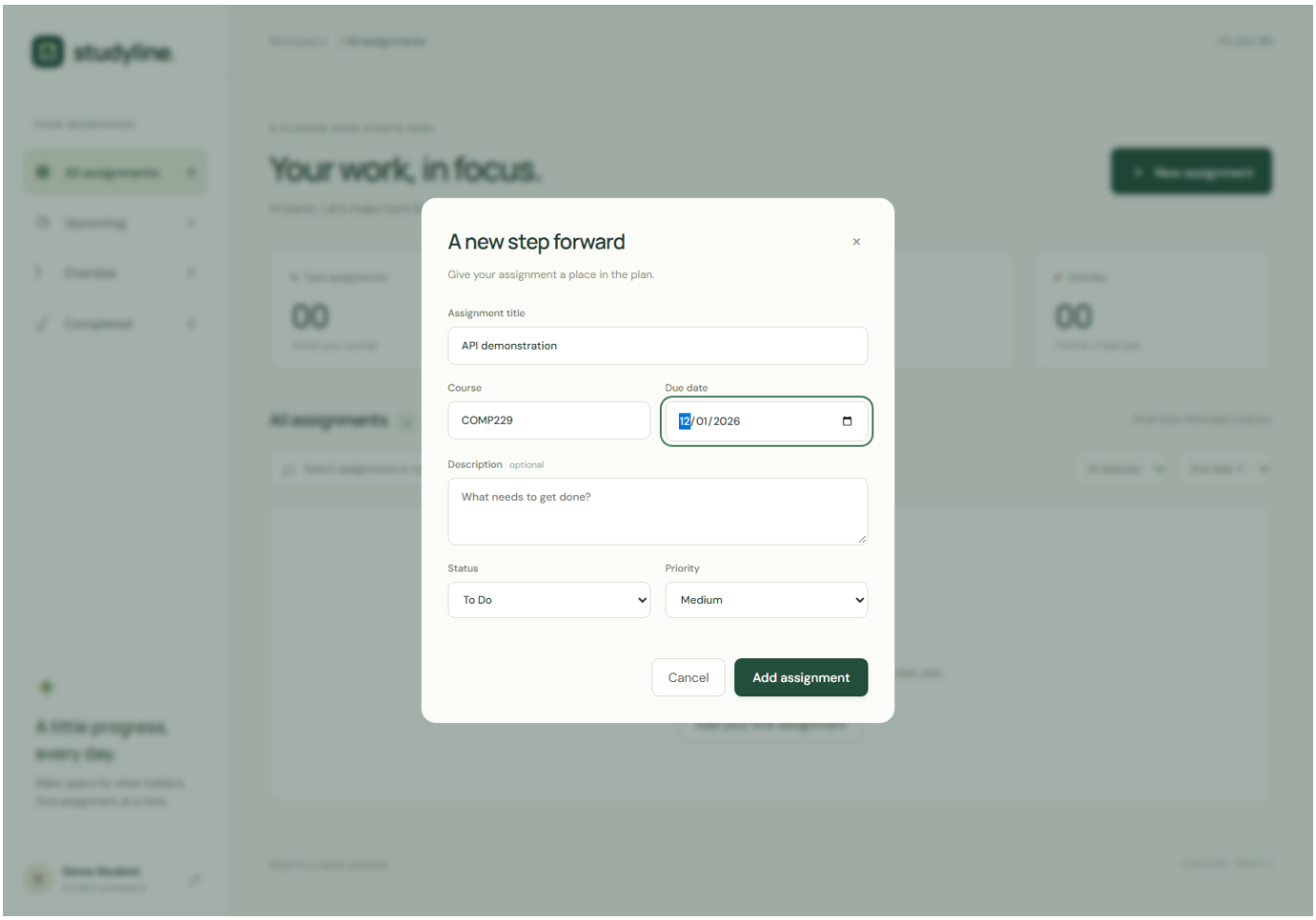
Sign in



Assignment dashboard



Assignment editor



6 Verification and release notes

Eight backend integration tests pass against a disposable real MongoDB instance. Coverage includes hashed passwords and sessions, duplicate accounts, CRUD persistence, partial-update preservation, cross-user access rejection, invalid payloads, CSRF defenses and rate limiting. The production Vite build succeeds. A direct connection from the application runtime to the existing Atlas cluster has been verified.

Browser verification

The browser workflow test exercises registration, creation, editing, completion, filtering, deletion and sign-out. See the accompanying test record for the latest result. Screenshots use generated demonstration accounts and example coursework, not real student records.

Remaining submission dependencies

The team must supply student numbers, real member photos and signatures, agree the final contract and duties, make genuine member commits, use an Agile board, and record/upload its demo. A prepared source package does not establish these human contributions or constitute a final submission.

Technical references

Express routing: https://expressjs.com/en/guide/routing/
Mongoose validation: https://mongoosejs.com/docs/validation.html
Vite guide: https://vite.dev/guide/